

Inheritance and Operator Overloading

Rohit Goswami · 2026-10-05 · python · fortran · numpy · f2py · ramblings

One might expect a `ChargedParticle` to be a `Particle`. One might also expect `+` to add two of them. Fortran agrees on both counts. The argument starts in the wrapper.

Background

The [parameterized types post](#) is the one that went out before this. Finalizers and generic bindings are further back, in [that post](#). This one is the inheritance piece I had sitting behind them: type extension (`extends()`, F2018 7.5.7)[^fn:1] and operator overloading (`interface operator()`, F2018 7.5.11).

Same plan as the allocatable post. Wrappers by hand first, then what `f2py` generates. The code is the [post/f2py-inheritance-operators](#) tag.

Type inheritance via `extends()`

The Fortran types

Start small. `Particle` has `mass` and `charge`. `ChargedParticle` extends it and adds `spin`.

```
module inheritance_mod
  use iso_c_binding
  implicit none

  type :: Particle
    double precision :: mass
    integer :: charge
  end type Particle

  type, extends(Particle) :: ChargedParticle
    double precision :: spin
  end type ChargedParticle

contains
  subroutine scale_mass(p, factor)
    type(Particle), intent(inout) :: p
    double precision, intent(in) :: factor
    p%mass = p%mass * factor
  end subroutine
end module
```

A `ChargedParticle` has `mass` and `charge` from `Particle`, and its own `spin`. The standard calls that type extension (F2018 7.5.7.1).

Manual wrappers

One might expect the child to reuse the parent's getters. It can't. Each type gets its own wrappers, and the child reads the inherited members through its own layout[^fn:2]:

► [Full inheritance wrapper module \(click to expand\)](#)

`charged_get_mass` does `c_f_pointer` into `type(ChargedParticle)`, not `type(Particle)`, and then reads `obj%mass`. The capsule holds a `ChargedParticle`. Reading that memory as a `Particle` works on every compiler I have tried^[^fn:2], and the standard does not promise it, so the wrappers stay type-specific.

Testing from C

► C test (click to expand)